FLOW CONTROL BENCHMARK EVIDENCE · GPT-OSS 20B · ONE GPU

Flow control makes shared inference predictable.

Flow control is the Endpoint Picker's policy layer for multi-tenant inference. While the GPU has available capacity, requests move through without queueing. When the pool is under pressure, flow control activates. New work waits in policy-aware queues before it enters vLLM's local queue, so priority, fairness, and tenant rules decide what runs next.

This benchmark answers the two questions platform teams care about. Can we consolidate workloads onto one GPU, raise utilization, and still uphold our latency SLOs? And when load exceeds capacity, does the platform keep priority workloads fast while lower-priority traffic absorbs the queue? Scenario 1 answers the first at an operating point with headroom. Scenarios 2 through 4 push the pool past saturation to answer the second.

Results at a glance

1 · CONSOLIDATION

114 to 125 ms

p95 TTFT, target 300 ms

Two tenants share one GPU. Both hold the target in every counted repeat, all requests HTTP 200.

2 · SERVICE TIERS

41 vs 379 ms

p50 TTFT, premium vs standard

Standard surges past saturation. Premium stays ahead of the queue. Standard absorbs the wait.

3 · FAIRNESS

382 vs 214 ms

p50 TTFT, spiker vs peer average

Same priority band. The spiking tenant carries the queue it creates. Peers stay bounded.

4 · BATCH ISOLATION

202 vs 64 ms

mean queue time, batch vs premium

Batch ramps to triple the interactive load. Interactive lanes stay ahead and batch absorbs the waiting.

Scenario 1 · Consolidation without giving up latency

Scenario 1 is the consolidation case. Two interactive workloads that would each hold their own GPU share one instead.

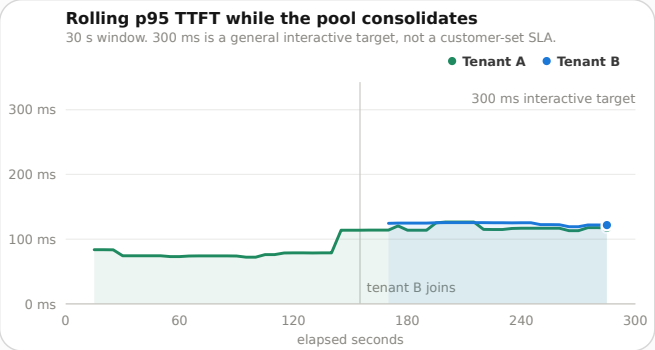
NORMAL OPERATING POINT

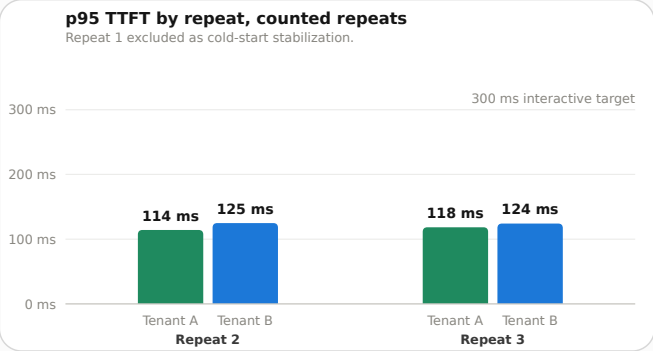
Two priority tenants on one GPU

✓ **Accepted evidence**

What it proves Two interactive workloads can share one GPU and uphold an interactive latency target. We used 300 ms p95 TTFT as a general working target.

What we saw Tenant A ran alone for the first half of the window and tenant B joined mid-run. The p95 TTFT stayed at 114 to 125 ms for both tenants in both counted repeats, and every request returned HTTP 200.





Every point in the left chart is one request from the counted run. Adding the second tenant moved the p95 TTFT from about 75 ms to about 120 ms, inside the 300 ms target. The first measured pass is excluded as cold stabilization. Both counted repeats agree.

Why it matters Consolidation is a cost decision. This operating point shows the shared GPU upholds the target with measured evidence, and flow control is the insurance if a spike pushes the pool past capacity.

Latency rerun, 2026-07-23 · 1 stabilization pass + 2 counted repeats · 1,298 counted requests, all HTTP 200 · 512-token prompts, 64-token responses

Scenario 2 · Priority under mixed load

Scenario 2 is the overload case that motivates service tiers. Premium tenants hold a steady interactive load while standard traffic surges past what the GPU can absorb.

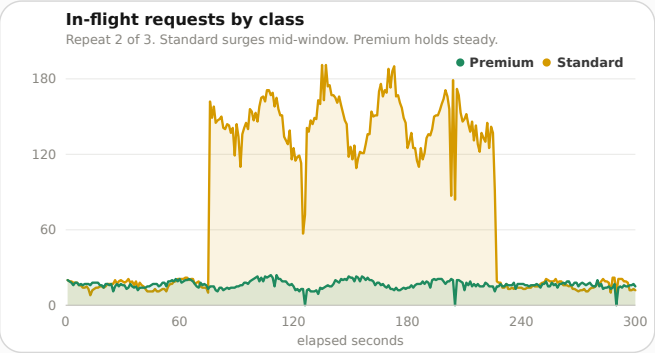
PRIORITY DIFFERENTIATION

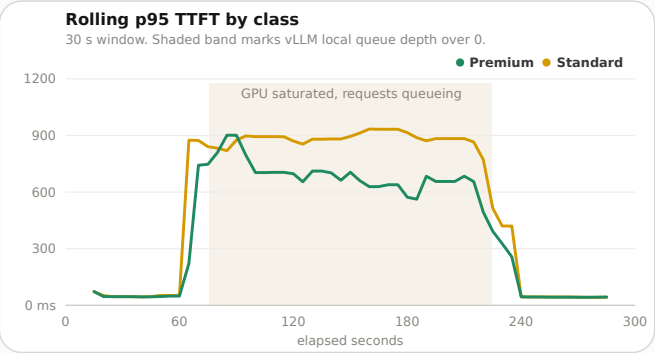
Premium steady, standard surge

✓ **Accepted evidence**

What it proves When the pool saturates, priority decides who waits. Premium traffic stays ahead while standard traffic absorbs the queue.

What we saw The surge saturated the GPU and requests queued. The p50 TTFT for premium tenants averaged 41 ms while the p50 TTFT for standard tenants averaged 379 ms. Priority, not arrival order, decided who waited.





During the shaded window the runtime reached its active-sequence limit and requests queued. The standard p95 TTFT rose toward 900 ms while the premium p95 settled near 550 to 650 ms after a brief transient at surge onset. Across all three repeats the premium p95 averaged 561 ms and the standard p95 averaged 895 ms.

Why it matters A latency-sensitive product keeps its experience through a surge it did not cause. The tier boundary holds under real pressure, not just on paper. That is what makes internal chargeback and SLA commitments enforceable on shared capacity.

Noisy priority run, 2026-07-24 · 3 counted repeats · 53,399 requests, 2 non-200 (two 503s on standard in repeat 2) · 512-token prompts, 128-token responses

Scenario 3 · Fairness inside one priority band

All three tenants are premium, so priority is not the differentiator here. Fairness is. Tenant A spikes repeatedly, then returns.

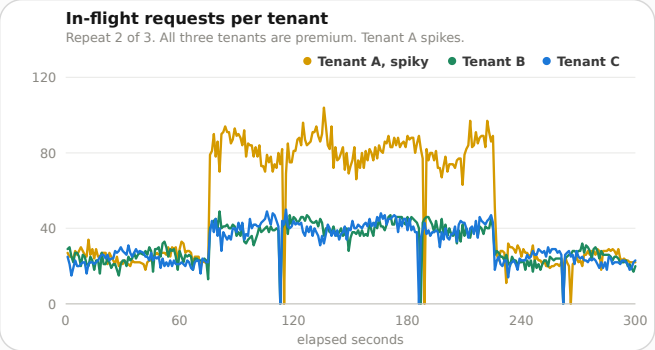
FAIRNESS INSIDE ONE BAND

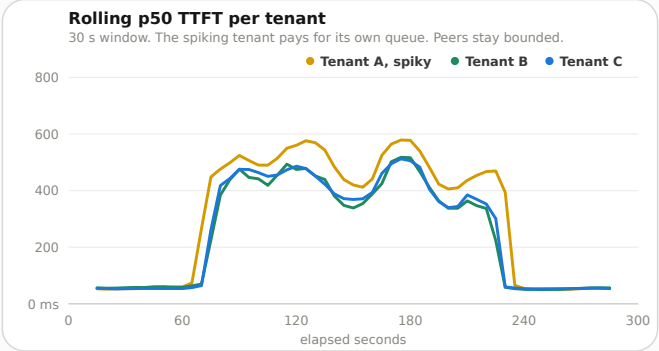
One premium tenant spikes

✓ Accepted evidence

What it proves Within one priority band, fairness bounds the damage a spiking tenant can do to its peers.

What we saw The spiking tenant carried a p50 TTFT of 382 ms while its peers stayed at 206 and 221 ms. The gap is moderate by design. Fairness bounds peers without punishing the spiker.





TENANT	TRAFFIC ROLE	AVG P50 TTFT	AVG P95 TTFT	AVG EPP QUEUE MEAN
premium-tenant-a	Greedy spike	382 ms	744 ms	107 ms
premium-tenant-b	Same-priority peer	206 ms	671 ms	80 ms
premium-tenant-c	Same-priority peer	221 ms	675 ms	81 ms

Why it matters Teams at the same priority can share a pool without a noisy neighbor starving them, and without anyone tuning per-tenant rate limits by hand. The platform absorbs the operational work that per-team quotas would create.

Saturated fairness run, 2026-07-24 · 1 stabilization pass + 3 counted repeats · 65,003 requests, all HTTP 200 · vLLM waiting p95 31 to 33 requests throughout

Scenario 4 · Batch isolation under surge

Scenario 4 is the case that makes consolidation safe to run hot. Batch traffic ramps into the service while interactive traffic stays present.

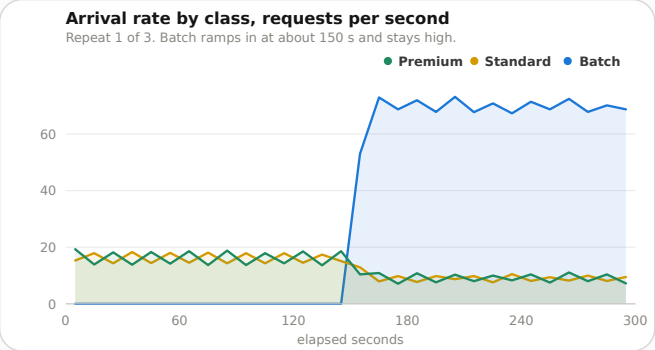
PRIORITY INVERSION PREVENTION

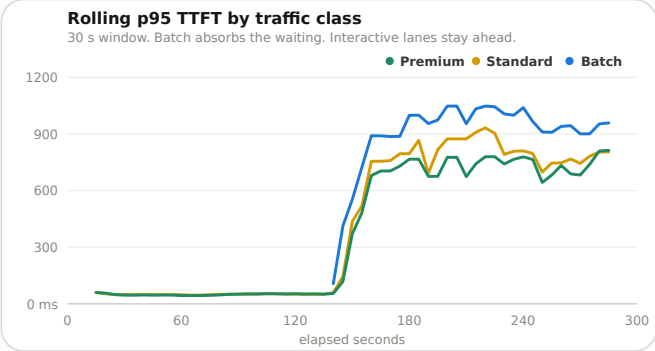
Batch ramps while interactive stays present

✓ Accepted evidence

What it proves Batch traffic cannot displace interactive traffic, even when batch dominates the arrival rate.

What we saw Batch ramped to about three times the interactive arrival rate. The mean queue time for batch averaged 202 ms while premium and standard stayed at 64 and 66 ms.





MEAN EPP QUEUE TIME, AVERAGE OF 3 REPEATS

Premium	64 ms
Standard	66 ms
Batch	202 ms

Batch carried roughly three times the queue time of the interactive lanes. In one repeat, two client-observed 503s appeared on batch traffic only. Premium and standard had no non-200 responses.

Why it matters Spare capacity can carry batch work during low-traffic periods without risking interactive SLOs. Overnight document and report pipelines can fill the same GPUs that serve interactive traffic by day. That is what lets a consolidated pool run at high utilization.

A separate overload probe with a tighter request time-to-live and queue budget shows controlled rejection under extreme pressure. Treat that as overload behavior, not the main production configuration.

Clean pressure pass, 2026-07-21 · 3 counted repeats · 53,954 requests, 2 non-200 (batch only) · 512-token prompts, 128-token responses

Policy auditability

You can verify the policy is real from four records in four layers, from the request header to the GPU runtime. The same endpoint serves multiple traffic classes because the gateway tags each request and the Endpoint Picker applies an explicit policy.

LAYER	WHAT IT RECORDS	WHY IT MATTERS
Request headers	`x-gateway-inference-objective`, `x-gateway-inference-fairness-id`	Traffic is classified by trusted platform policy, not by a hidden route.
InferenceObjective	Premium 100, standard 0, batch -10	Business priority maps to a concrete platform resource.
Endpoint Picker queues	Priority band, fairness ID, queue duration	Queueing can be explained by tenant and traffic class.
vLLM metrics	Running and waiting requests	Shows when the GPU runtime is actually saturated.

Methodology

Setup

- Model: GPT-OSS 20B on one GPU, served by vLLM behind the inference gateway and Endpoint Picker
- Priority bands: premium 100, standard 0, batch -10, round-robin fairness within a band
- Saturation gate: pluggable detectors on queue depth and KV-cache utilization
- Synthetic prompts targeting 512 input tokens, with fixed response lengths per scenario

Measurement

- Traffic for the saturation scenarios is noisy and sinusoidal on purpose, closer to production arrival patterns than a flat synthetic load. Each scenario's traffic chart shows the pattern that ran
- TTFT measured client side, from request start to first streamed token, through the gateway
- Each scenario runs 300 s of active traffic, repeated 3 times. Warmup and stabilization passes are excluded
- Every request is logged individually; all charts here are drawn from that per-request data
- Fixed response lengths are a benchmark control. Production responses end at natural completion, so end-to-end latency figures from these runs are not production predictions. TTFT is the comparable metric.

How to read this evidence

The four scenarios come from separate benchmark campaigns, each with its own accepted run, and each provenance line above names its run. Use Scenario 1 for the consolidation claim. Use Scenarios 2 through 4 for behavior under pressure. The pattern is the finding. At the consolidated operating point the latency target held, and past saturation the policy decided who waited, by priority, by fairness, and by traffic class.